



COMPANY OR INSTITUTION NAME

Department, Team or Reference Sector

Technical Report / Project Report

Project Reference, Client or Exam

EngTeX — Engineering L^AT_EX

Template

User Manual

Author:

Luca Gaetano Sansalone

Version:

Version 1.1

Date: June 30, 2026

Contents

1	Introduction and Architecture	1
1.1	Modular Document Structure	1
1.1.1	The "notikz" Mode (Overleaf Free)	1
1.2	Basic Graphic Boxes	2
1.2.1	Definitions	2
1.2.2	Notes and Warnings	2
1.2.3	Numbered Examples	2
1.3	Study and Notes Boxes	2
1.3.1	Reminders and Tips	2
1.3.2	Exam Questions and Fun Facts	3
2	Math and Theorems	4
2.1	Theoretical Environments (Theorems)	4
2.2	Advanced Mathematical Macros	5
2.2.1	Calculus and Derivatives	5
2.2.2	Linear Algebra and Vectors	5
2.2.3	Vector Operators (Physics and Electromagnetism)	5
2.2.4	Probability and Statistics	5
2.2.5	Number Sets and Phasors	6
2.3	Equations and Alignments	6
3	Advanced Tools (Code and Graphics)	7
3.1	Source Code and Scripting	7
3.1.1	Inline code in the text	8
3.2	Pseudocode and Algorithms	8
3.3	Electrical Circuits	9
3.4	Function Plots	9
3.5	Block Diagrams (TikZ)	9
4	Quick Command Reference	11
4.1	Document Customisation	11
4.2	Graphic Boxes	12
4.3	Math Macros	13
4.4	Physical Units	13
4.5	Code Listings	13
4.6	TikZ Diagram Macros	14

Chapter 1

Introduction and Architecture

This chapter describes the modular architecture of the `engtex.sty` package and introduces the basic formatting tools: definition boxes, warning boxes, and study notes boxes.

1.1 Modular Document Structure

The document is split into separate `.tex` files, one for each chapter, collected in the `capitoli/` folder. The `main_ENG_Manual.tex` file acts as the orchestrator: it defines the preamble, acronyms, and calls the chapters in the desired order using `\input`.

Behind the scenes, the `layout/engtex.sty` file acts as a "conductor" for the various thematic modules:

- `engtex-math.sty`: Advanced math macros
- `engtex-boxes.sty`: Theorems and visual boxes
- `engtex-code.sty`: Syntax highlighting and listings
- `engtex-algorithms.sty`: Pseudocode
- `engtex-circuits.sty`: Electrical circuits
- `engtex-plots.sty`: Function plotting

You will never need to load them individually, just include `engtex` in your preamble.

1.1.1 The "notikz" Mode (Overleaf Free)

Some advanced packages (like `circuitikz` and `pgfplots`) require a lot of memory to compile complex graphics. If you use the free tier of Overleaf, you might encounter a *Timeout* error.

To solve this issue during the drafting phases, you can temporarily disable heavy graphics by declaring in the preamble:

```
\usepackage[notikz]{layout/engtex}
```

This command ensures that all electrical circuits, plots, and block diagrams are ignored during compilation, drastically reducing wait times.

1.2 Basic Graphic Boxes

The template provides several graphic environments to visually structure technical content. All boxes are managed via `tcolorbox`.

1.2.1 Definitions

To insert a formal definition with a light blue background, type:

```
\definition{Definition Title}{Definition text...}
```

Definition: Laplacian Filter

The Laplacian filter is a second-order differential operator used in image processing for edge detection.

1.2.2 Notes and Warnings

To insert a warning or a technical note, use the grey sidebar by typing `\note{...}`:

Note: *Ensure that the sampling frequency satisfies the Nyquist-Shannon theorem.*

To highlight a common error, a critical constraint, or an unexpected behavior that the user must keep in mind, use the `\warning{...}` box (red sidebar):

Warning: Do not insert the `\node` command before the integrated TikZ macros.

1.2.3 Numbered Examples

To insert a practical example. The template will automatically number the box based on the current chapter (e.g. Example 1.1).

```
\example{Optional Title}{Your example text...}
```

Example 1.1 – Voltage Divider Calculation

If we have $V_{in} = 5\text{ V}$, $R_1 = 1\text{ k}\Omega$ and $R_2 = 2\text{ k}\Omega$, the output voltage will be $V_{out} = 3.33\text{ V}$.

1.3 Study and Notes Boxes

Ideal for university lecture transcriptions or study notes, these boxes allow you to highlight additional information without cluttering the main text.

1.3.1 Reminders and Tips

Use `\remember{Text}` for a yellow box highlighting key formulas:

Remember

Reactive power is measured in VAR, not in Watts.

Use `\tip{Text}` for a green box with a useful hint:

Tip

In approximate calculations, remember to keep at least three significant figures until the final result.

1.3.2 Exam Questions and Fun Facts

To isolate concepts frequently asked during oral exams, use `\examquestion{Text}`:

Exam Question

Prove the validity of Thevenin's Theorem for linear active networks.

Finally, for anecdotes or interesting facts, use `\curiosity{Text}` (in magenta color):

Fun Fact

The VHDL language was initially developed on behalf of the US Department of Defense in 1983.

***Note:** All commands have their Italian equivalent (e.g. `\definizione`, `\attenzione`, `\curiosita`). If you pass the `[italian]` option to the package, the printed labels ("Example" $\rightarrow$ "Esempio") will also change automatically.*

Chapter 2

Math and Theorems

The STEM fields (Science, Technology, Engineering, and Mathematics) require advanced tools to write formulas and present theory rigorously. The `engtex-math.sty` and `engtex-boxes.sty` modules significantly extend the native capabilities of L^AT_EX.

2.1 Theoretical Environments (Theorems)

For transcribing mathematical notes, the template offers five dedicated boxes, equipped with automatic numbering linked to the chapter.

```
\theorem{Theorem Name}{Theorem statement...}
\corollary{Corollary Name}{Corollary text...}
\lemma{Lemma Name}{Lemma text...}
\proposition{Proposition Name}{Proposition text...}
```

Theorem 2.1: Gauss-Markov Theorem

Under the assumptions of the classical linear model, the ordinary least squares (OLS) estimator is the best linear unbiased estimator (BLUE).

Corollary 2.1: Existence of minimum variance

If errors are homoscedastic, no other linear unbiased estimator can have a lower variance than the OLS estimator.

Proposition: Practical Proposition

Proofs can be inserted without altering the progressive numbering of theorems.

For formal proofs, a specific environment is provided, without a background but with a thick sidebar and an automatic concluding QED symbol (■):

```
\proofenv{Let's write all the mathematical steps here...}
```

Proof:

From the statement we derive that $A = B$. Since $B = C$, by the transitive property of equality it logically follows that $A = C$. This concludes the proof and proves the initial assertion in every analyzed case. ■

2.2 Advanced Mathematical Macros

The package introduces various shortcuts to type complex formulas faster.

2.2.1 Calculus and Derivatives

Writing derivatives and integrals in L^AT_EX requires a lot of code. Use these abbreviations:

- First derivative: `\der{f}{x}` $\rightarrow \frac{df}{dx}$
- Partial derivative: `\pder{f}{y}` $\rightarrow \frac{\partial f}{\partial y}$
- Multiple derivative: `\dder{f}{x}` $\rightarrow \frac{d^2 f}{dx^2}$
- n -th derivative: `\nder{n}{f}{x}` $\rightarrow \frac{d^n f}{dx^n}$
- Definite integral: `\intab{a}{b}{x^2}{x}` $\rightarrow \int_a^b x^2 dx$
- Limit: `\limit{x}{\infty} f(x)` $\rightarrow \lim_{x \rightarrow \infty} f(x)$
- Summation: `\summ{i=1}{n} i` $\rightarrow \sum_{i=1}^n i$

2.2.2 Linear Algebra and Vectors

Vectors and matrices enjoy simplified commands:

- Module/Norm: `\norm{v}` $\rightarrow \|v\|$
- Absolute value: `\abs{x}` $\rightarrow |x|$
- Dot/Inner product: `\inner{u}{v}` $\rightarrow \langle u, v \rangle$
- Trace, Rank, Adjugate: `\tr{A}`, `\rank{B}`, `\adj{C}` $\rightarrow \text{tr}(A), \text{rk}(B), \text{adj}(C)$
- Fast 3×3 matrix: `\matrixthree{1}{2}{3}{4}{5}{6}{7}{8}{9}`

$$M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

2.2.3 Vector Operators (Physics and Electromagnetism)

Particularly useful in Engineering Physics or Telecommunications:

- Gradient: `\grad V` $\rightarrow \nabla V$
- Divergence: `\dive \mathbf{E}` $\rightarrow \nabla \cdot \mathbf{E}$
- Curl: `\curl \mathbf{H}` $\rightarrow \nabla \times \mathbf{H}$
- Laplacian: `\lap f` $\rightarrow \nabla^2 f$

2.2.4 Probability and Statistics

- Probability: `\prob{A|B}` $\rightarrow P(A|B)$
- Expected value: `\expval{X}` $\rightarrow E[X]$
- Variance and Covariance: `\var{X}`, `\cov{X}{Y}` $\rightarrow \text{Var}(X), \text{Cov}(X, Y)$

2.2.5 Number Sets and Phasors

- Sets: $\backslash\mathbb{N}$, $\backslash\mathbb{Z}$, $\backslash\mathbb{Q}$, $\backslash\mathbb{R}$, $\backslash\mathbb{C} \rightarrow \mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{C}$
- Phasors (EE): $\backslash\text{fasore}\{V\}\{\backslash\theta\} \rightarrow V\angle\theta$
- Real and imaginary parts: $\backslash\text{real}\{z\}$, $\backslash\text{imag}\{z\} \rightarrow \Re\{z\}, \Im\{z\}$
- Complex conjugate: $\backslash\text{conj}\{z\} \rightarrow \bar{z}$

2.3 Equations and Alignments

Instead of always writing `\begin{equation} ... \end{equation}`, you can use the fast wrapper:

`\eqn{eq:mass}\{E = mc^2\}`

$$E = mc^2 \tag{2.1}$$

For systems or multi-line equations:

```
\eqns{eq:system}{
  x &= 2y + 1 \\
  y &= x - 3
}
```

$$x = 2y + 1 \tag{2.2}$$

$$y = x - 3 \tag{2.3}$$

All these environments can be referenced with `\ref{...}`, for example Eq. [2.2](#).

Chapter 3

Advanced Tools (Code and Graphics)

In this chapter we explore the integration of source code, pseudocode, and technical graphics (schematics and plots).

3.1 Source Code and Scripting

The `engtex-code.sty` package includes preconfigured settings based on the `listings` package for the most common languages: C, C++, Java, Python, MATLAB, VHDL, Bash, and LaTeX.

The general syntax for code blocks is:

```
\begin{lstlisting}[style=python, caption={Example script}]
def greeting():
    print("Hello World")
\end{lstlisting}
```

```
1 def greeting():
2     print("Hello World")
```

Listing 3.1: Example script

```
1 public class Main {
2     public static void main(String[] args) {
3         System.out.println("Constructor executed.");
4     }
5 }
```

Listing 3.2: Java Example

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity DFF is
5     Port ( D : in STD_LOGIC;
6           CLK : in STD_LOGIC;
7           Q : out STD_LOGIC );
8 end DFF;
```

Listing 3.3: VHDL Example

The following keywords are available for the `style=...` parameter:

- `c`, `cpp`, `java` (C-like syntax, object-oriented)

- `python`, `matlab` (Numerical scripting and data science)
- `vhdl` (Hardware description)
- `bash` (Terminal and system commands, with dark background)
- `latex` (TeX syntax)

3.1.1 Inline code in the text

To cite variables or small instructions within a paragraph, use the *inline* commands:

- `\cinline{int x = 0;}`
- `\cppinline{std::cout << x;}`
- `\pythoninline{import numpy as np}`
- `\javainline{public static void main}`
- `\matlabinline{zeros(3,3)}`
- `\vhdlinline{rising_edge(clk)}`

Here are some practical examples of *inline* code:

In Python, to calculate the mean we use `np.mean(data)`.

In Java, the entry point is `public static void main(String[] args)`.

In VHDL, the rising edge detection is done with `rising_edge(clk)`.

3.2 Pseudocode and Algorithms

If you need to describe the logic of a program regardless of the language, you can use the `algorithm` environment, provided by `engtex-algorithms.sty` (based on `algorithm2e`).

```
\begin{algorithm}[H]
  \SetAlgoLined
  \KwData{an array  $A$ }
  \KwResult{sum of elements}
   $S \leftarrow 0$ ;
  \For{$i \leftarrow 1$ \KwTo  $N$ }{
     $S \leftarrow S + A[i]$ ;
  }
  \Return{$S$};
  \caption{Sum Calculation}
\end{algorithm}
```

Algorithm 1: Sum Calculation

Data: an array A

Result: sum of elements

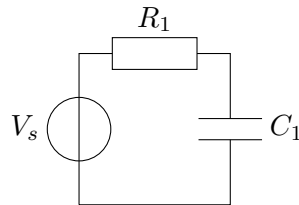
```
1  $S \leftarrow 0$ ;
2 for  $i \leftarrow 1$  to  $N$  do
3   |  $S \leftarrow S + A[i]$ ;
4 end
5 return  $S$ ;
```

All keywords ("Data", "Result") are automatically translated if the document is set to `italian`.

3.3 Electrical Circuits

For Electrical and Electronic Engineering, `engtex-circuits.sty` pre-loads `circuitikz` with European and compact styles.

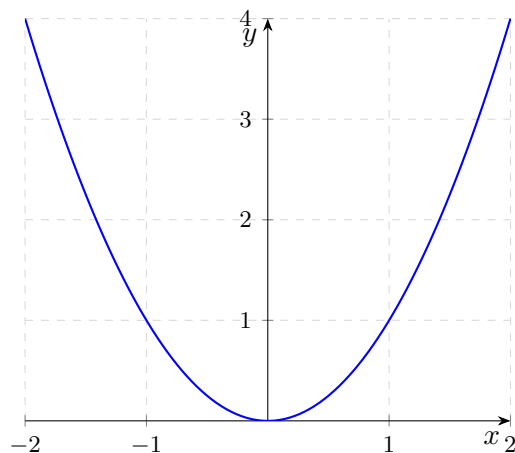
```
\begin{center}
\begin{circuitikz}
    \draw (0,0) to[V, l=$V_s$] (0,2)
           to[R, l=$R_1$] (2,2)
           to[C, l=$C_1$] (2,0) -- (0,0);
\end{circuitikz}
\end{center}
```



3.4 Function Plots

To represent data and functions, `engtex-plots.sty` uses `pgfplots`. It is pre-set with a style called `engtex` plot that guarantees clean grids and optimal proportions.

```
\begin{center}
\begin{tikzpicture}
    \begin{axis}[engtex plot, xlabel=$x$, ylabel=$y$, width=8cm]
        \addplot[blue, thick, domain=-2:2] {x^2};
    \end{axis}
\end{tikzpicture}
\end{center}
```

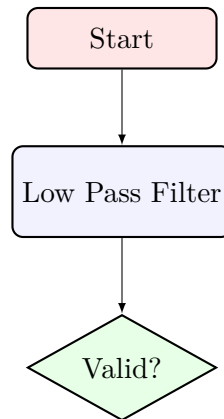


3.5 Block Diagrams (TikZ)

The classic `engtex-tikz.sty` automation macros allow you to generate blocks without remembering the entire TikZ syntax:

- `\startendblock{id}{Text}{positioning}`

- `\sysblock{id}{Text}{positioning}`
- `\flowblock{id}{Text}{positioning}`
- `\decblock{id}{Text}{positioning}`



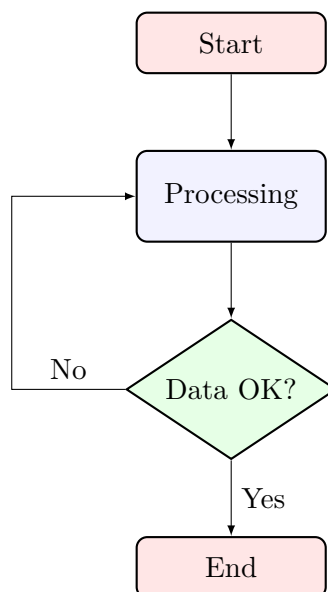
Here is a slightly more complex example with a feedback loop:

```

\begin{center}
\begin{tikzpicture}
  \startendblock{start}{Start}{};
  \sysblock{proc}{Processing}{below=1cm of start};
  \decblock{cond}{Data OK?}{below=1cm of proc};
  \startendblock{end}{End}{below=1cm of cond};

  \draw[-latex] (start) -- (proc);
  \draw[-latex] (proc) -- (cond);
  \draw[-latex] (cond) -- node[right] {Yes} (end);
  % Feedback loop
  \draw[-latex] (cond.west) -- node[above] {No} ++(-1.5,0) |- (proc.west);
\end{tikzpicture}
\end{center}

```



Chapter 4

Quick Command Reference

This chapter collects all template commands in tabular form, organised by category. Use it as a quick reference while writing your document, without consulting previous chapters.

4.1 Document Customisation

Command	Where to use
<code>\renewcommand{\versiondoc}{X.Y}</code> Sets the document version shown on the cover page.	<code>main.tex</code>
<code>\viewversion</code> Prints <i>Version X.X</i> or <i>Versione X.X</i> at the call site, depending on the package language option.	Cover page / header
<code>\DeclareAcronym{key}{short=..., long=...}</code> Declares an acronym.	<code>main.tex</code> preamble
<code>\ac{key}</code> Inserts the acronym (expanded on first use).	Chapter text
<code>\input{capitoli/file.tex}</code> Includes a chapter file.	<code>main.tex</code> chapters section
<code>\cite{key}</code> Inserts a numeric IEEE citation.	Chapter text

Table 4.1: Document customisation commands.

4.2 Graphic Boxes

Command (EN)	Style	Typical use
<code>\definition{Title}{Text}</code>	Blue box with title	Formal definitions, key concepts
<code>\note{Text}</code>	Grey-blue lateral bar	Technical notes, cross-references
<code>\warning{Text}</code>	Red lateral bar	Common errors, critical constraints
<code>\example{Title}{Text}</code>	Teal numbered box	Worked examples, calculations
<code>\theorem{Title}{Text}</code>	Purple numbered box	Theorems, corollaries, lemmas
<code>\proofenv{Text}</code>	No background, QED	Mathematical proofs
<code>\remember{Text}</code>	Yellow box	Key formulas to remember
<code>\examquestion{Text}</code>	Dark orange box	Typical exam questions
<code>\curiosity{Text}</code>	Magenta box	Fun facts and anecdotes

Table 4.2: English box commands.

English	Italian equivalent
<code>\definition{Title}{Text}</code>	<code>\definizione{Titolo}{Testo}</code>
<code>\note{Text}</code>	<code>\nota{Testo}</code>
<code>\warning{Text}</code>	<code>\attenzione{Testo}</code>
<code>\example{Title}{Text}</code>	<code>\esempio{Titolo}{Testo}</code>
<code>\theorem{Title}{Text}</code>	<code>\teorema{Titolo}{Testo}</code>
<code>\proofenv{Text}</code>	<code>\dimostrazione{Testo}</code>
<code>\remember{Text}</code>	<code>\ricorda{Testo}</code>
<code>\examquestion{Text}</code>	<code>\domandaesame{Testo}</code>
<code>\curiosity{Text}</code>	<code>\curiosita{Testo}</code>

Table 4.3: EN/IT box command equivalents.

4.3 Math Macros

Command	Output	Category
<code>\matrixthree{a}{b}...{i}</code>	3×3 matrix	Linear algebra
<code>\system{eq1 \ eq2}</code>	Aligned system	Linear algebra
<code>\der{f}{x}, \pder{V}{t}</code>	$\frac{df}{dx}, \frac{\partial V}{\partial t}$	Differential calculus
<code>\dder{f}{x}, \nder{n}{f}{x}</code>	$\frac{d^2 f}{dx^2}, \frac{d^n f}{dx^n}$	Multiple derivatives
<code>\intab{a}{b}{f}{x}</code>	$\int_a^b f dx$	Integral calculus
<code>\eval{f}{a}{b}</code>	$[f]_a^b$	Integral calculus
<code>\fourier{x(t)}, \laplace{h(t)}</code>	$\mathcal{F}\{x(t)\}, \mathcal{L}\{h(t)\}$	Transforms
<code>\grad{V}, \dive{\mathbf{E}}</code>	$\nabla V, \nabla \cdot \mathbf{E}$	Vector operators
<code>\prob{E}, \expval{X}</code>	$P(E), \mathbb{E}[X]$	Stats and Probability
<code>\notlog{A}</code>	$\overline{A}$	Boolean algebra
<code>\real{Z}, \imag{Z}</code>	$\Re\{Z\}, \Im\{Z\}$	Complex numbers
<code>\fasore{V}{theta}</code>	$V \angle \theta$	Phasors
<code>\RR, \NN, \ZZ, \CC</code>	$\mathbb{R}, \mathbb{N}, \mathbb{Z}, \mathbb{C}$	Number sets
<code>\norm{x}</code>	$\ x\ $	Vector spaces

Table 4.4: Math macro reference.

4.4 Physical Units

Command	Output	Notes
<code>\qty{150}{\MHz}</code>	150 MHz	Simple unit
<code>\qty{4.2}{\ns}</code>	4.2 ns	Prefixed unit
<code>\qty{9.81}{\m\per\s\squared}</code>	9.81 m s ⁻²	Compound unit
<code>\qty{3.3}{\V}</code>	3.3 V	Volt
<code>\qty{1}{\kohm}</code>	1 kΩ	Ohm with prefix

Table 4.5: SIunitx command examples.

4.5 Code Listings

style=	Language	Notes
<code>vhdl</code>	VHDL	IEEE 1076-2008, 4 keyword levels
<code>c</code>	C	Universal light style
<code>cpp</code>	C++	Universal light style
<code>java</code>	Java	Universal light style
<code>python</code>	Python	Universal light style
<code>matlab</code>	MATLAB	Universal light style

Table 4.6: Available listing styles.

Basic syntax (same for all languages):

```
\begin{lstlisting}[style=python, caption={Listing description}]
% your code here
```

```
\end{lstlisting}
```

Inline command	Language
<code>\vhdlinline{...}</code>	VHDL
<code>\cinline{...}</code>	C
<code>\cppinline{...}</code>	C++
<code>\javainline{...}</code>	Java
<code>\pythoninline{...}</code>	Python
<code>\matlabinline{...}</code>	MATLAB

Table 4.7: Inline code commands.

4.6 TikZ Diagram Macros

Macro	Shape	Typical use
<code>\startendblock{id}{label}{pos}</code>	Red rounded rect.	Start / end of algorithm
<code>\sysblock{id}{label}{pos}</code>	Blue rounded rect.	Hardware blocks, registers
<code>\flowblock{id}{label}{pos}</code>	Orange rectangle	Software ops, processes
<code>\decblock{id}{label}{pos}</code>	Green diamond	Conditions, branches

Table 4.8: TikZ macro reference.

Recommended `tikzpicture` environment for flowcharts:

```
\begin{figure}[H]
\centering
\begin{tikzpicture}[node distance=1cm,
  arrow/.style={thick,->,>=stealth}]
% nodes
\startendblock{start}{Start}{};
\flowblock{step1}{Operation}{below=of start};
% arrows
\draw[arrow] (start) -- (step1);
\end{tikzpicture}
\caption{Diagram description.}
\end{figure}
```